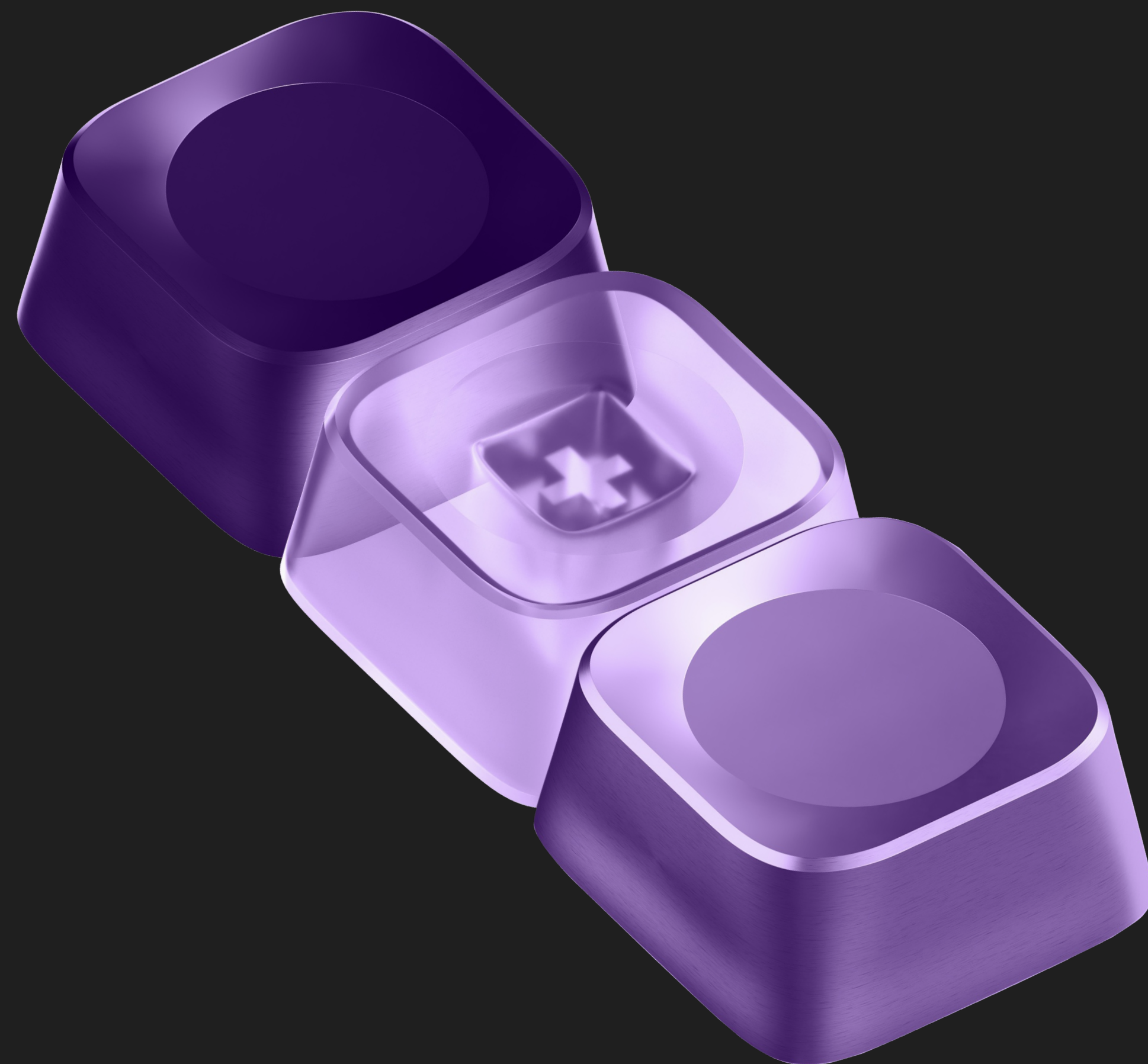




# Функции и процедуры в SQL

SQL И БАЗЫ ДАННЫХ

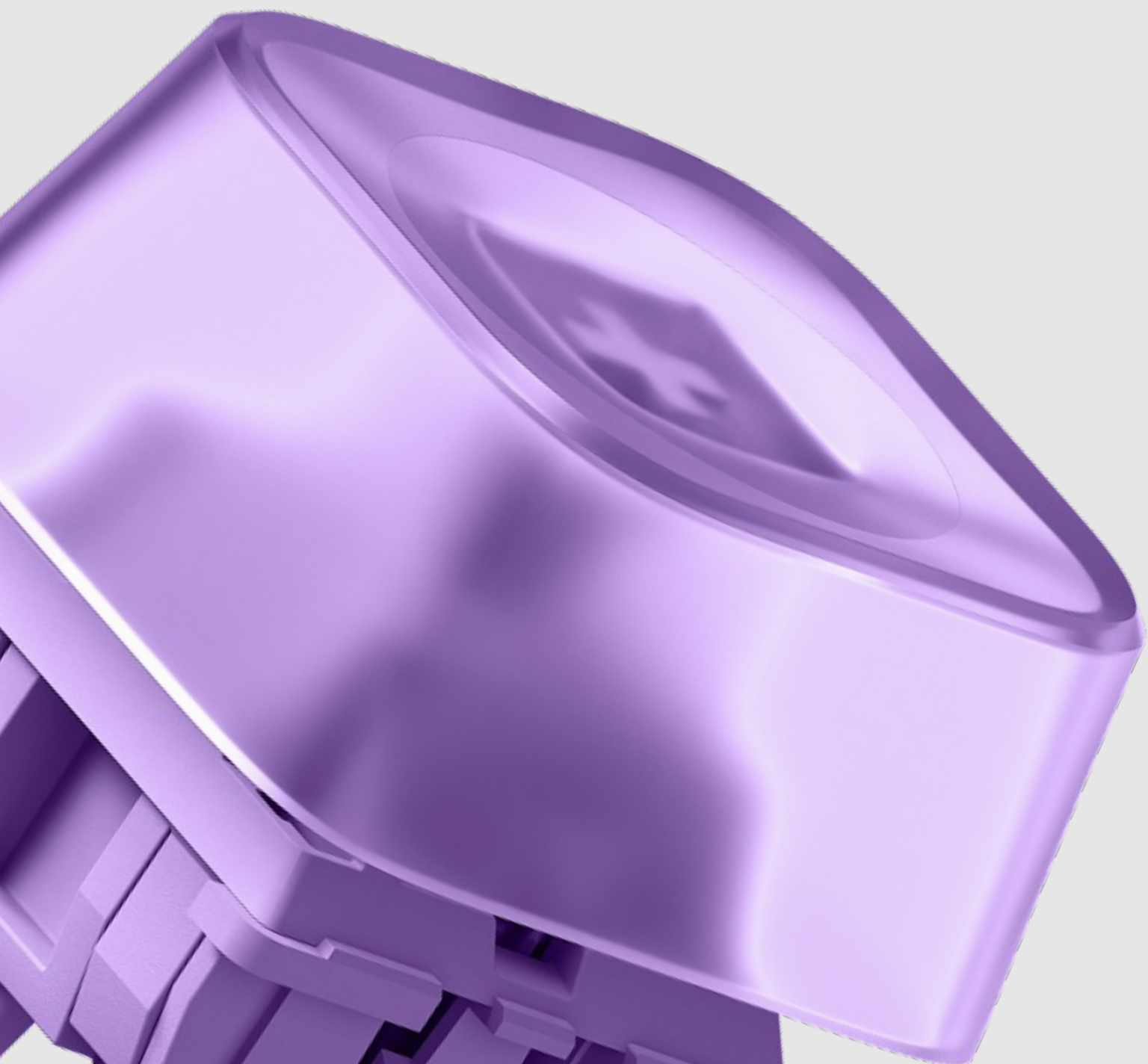
НЕДЕЛЯ 10



# ОСНОВЫ

SQL как язык  
программирования

Часть 1



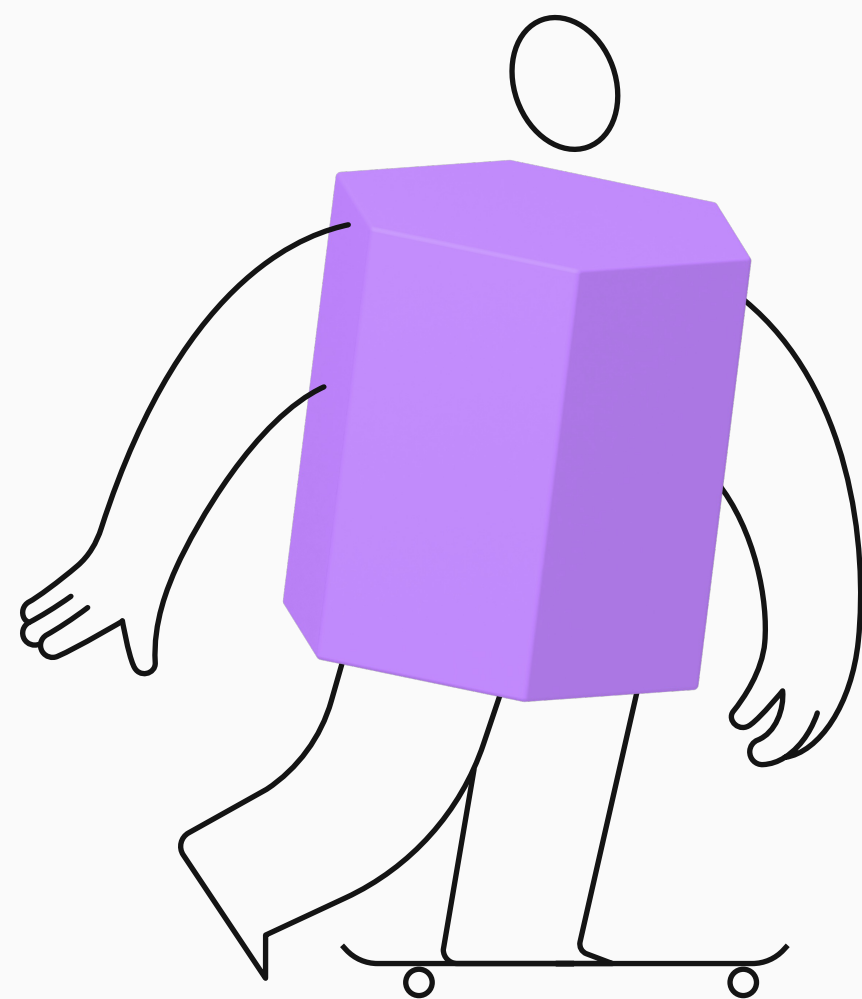
→ Отличие декларативного SQL от процедурного

→ Зачем писать код внутри базы данных

→ Знакомство с PL/pgSQL и анатомия кода

# Зачем нужен процедурный SQL?

SQL — декларативный язык.  
Чего ему не хватает?



- **Стандартный SQL:** говорим, ЧТО получить (например, `SELECT * FROM users`), но не говорим КАК.
- **Ограничения стандартного SQL:**
  - нет переменных;
  - нет условной логики (if/else в классическом виде программирования);
  - нет циклов (for, while);
  - сложно обрабатывать ошибки на лету.
- **Решение** — процедурный SQL: добавляет элементы императивного программирования прямо в СУБД.



# Архитектура: «умная» база данных

Перенос логики в БД: плюсы и минусы

## Преимущества

Снижение сетевого трафика

Инкапсуляция и безопасность

Централизация логики

## Недостатки

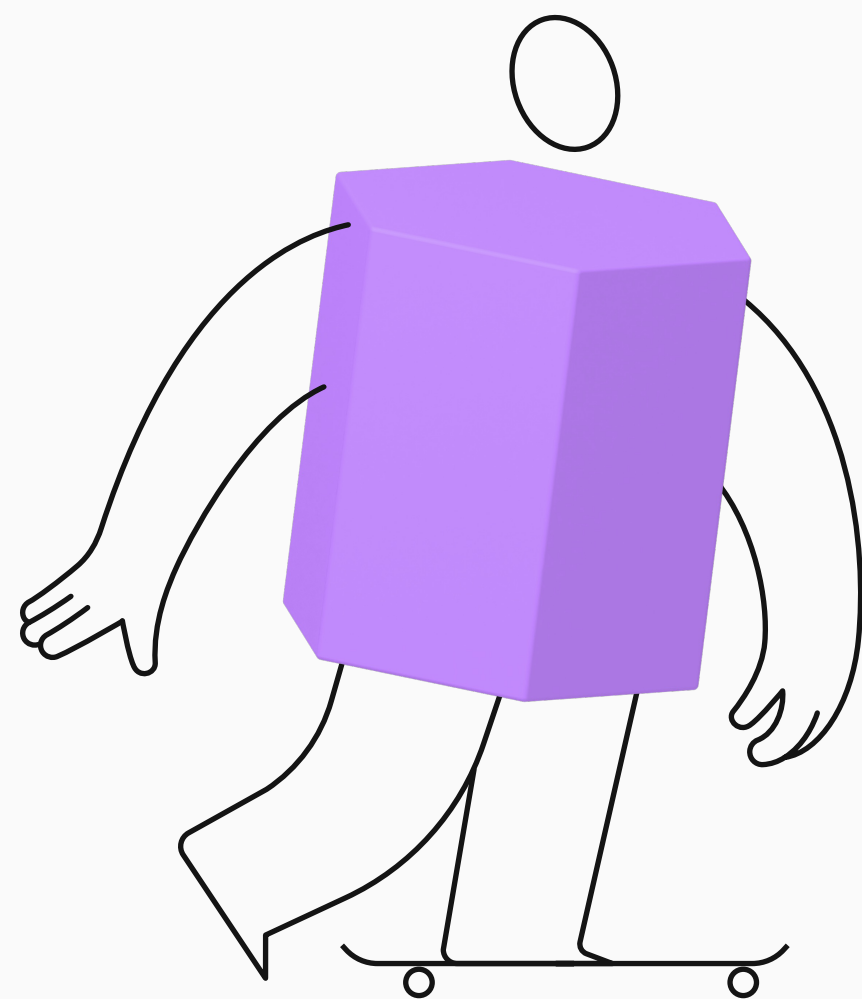
Сложность масштабирования

Vendor Lock-in



# PL/pgSQL

PostgreSQL поддерживает и другие языки. Можно писать функции на PL/Python, PL/Perl или даже PL/v8 (JavaScript).

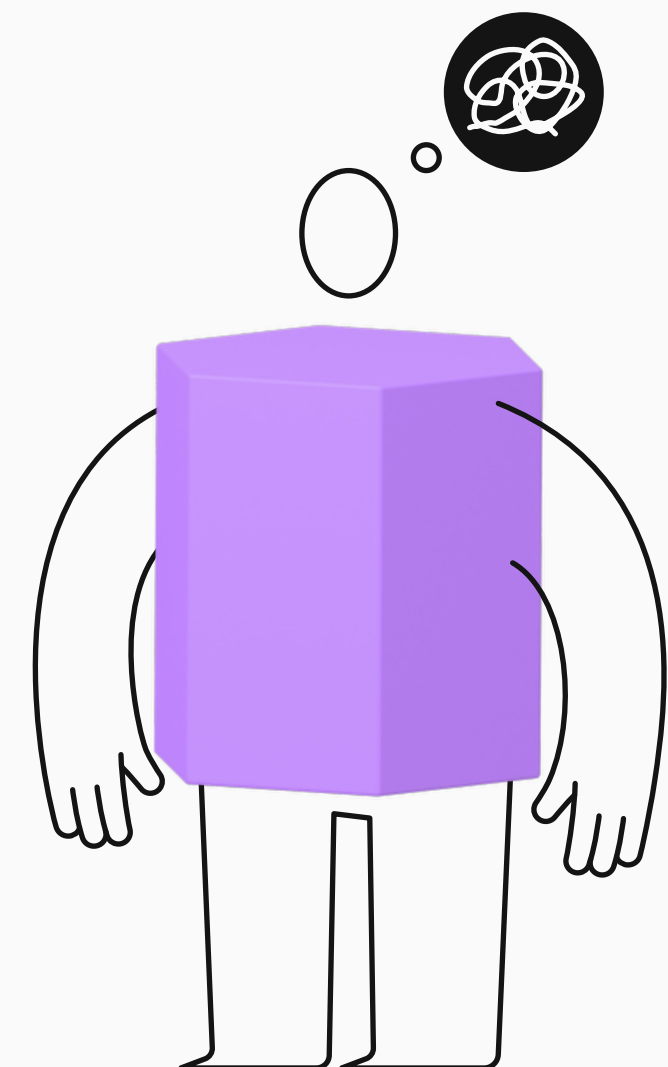


1. PL/pgSQL (Procedural Language/PostgreSQL) — загружаемое расширение по умолчанию.
2. Полностью интегрирован с типами данных PostgreSQL.
3. Является Тьюринг-полным языком (можно написать программу любой сложности).

# Структура блока кода

```
[<<Имя_блока>>]
[DECLARE
-- Список переменных
]
BEGIN
[EXCEPTION
-- Перехват ошибок
]
END [<<Имя_блока>>];
```

Весь код в PL/pgSQL делится на блоки, и он имеет строгую структуру.

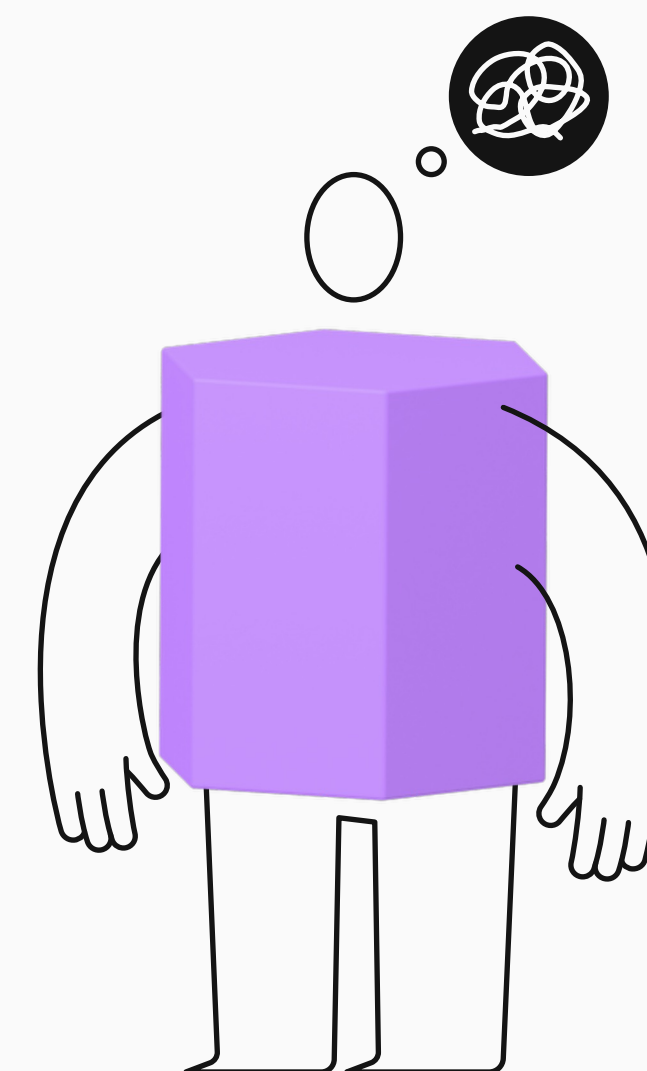


# Структура блока кода

```
DO $$  
DECLARE  
    my_name VARCHAR := 'student';  
BEGIN  
    RAISE NOTICE 'Hello, %! This is my  
first code!', my_name;  
END;  
$$;
```

Для тестов и разовых скриптов используется команда DO.

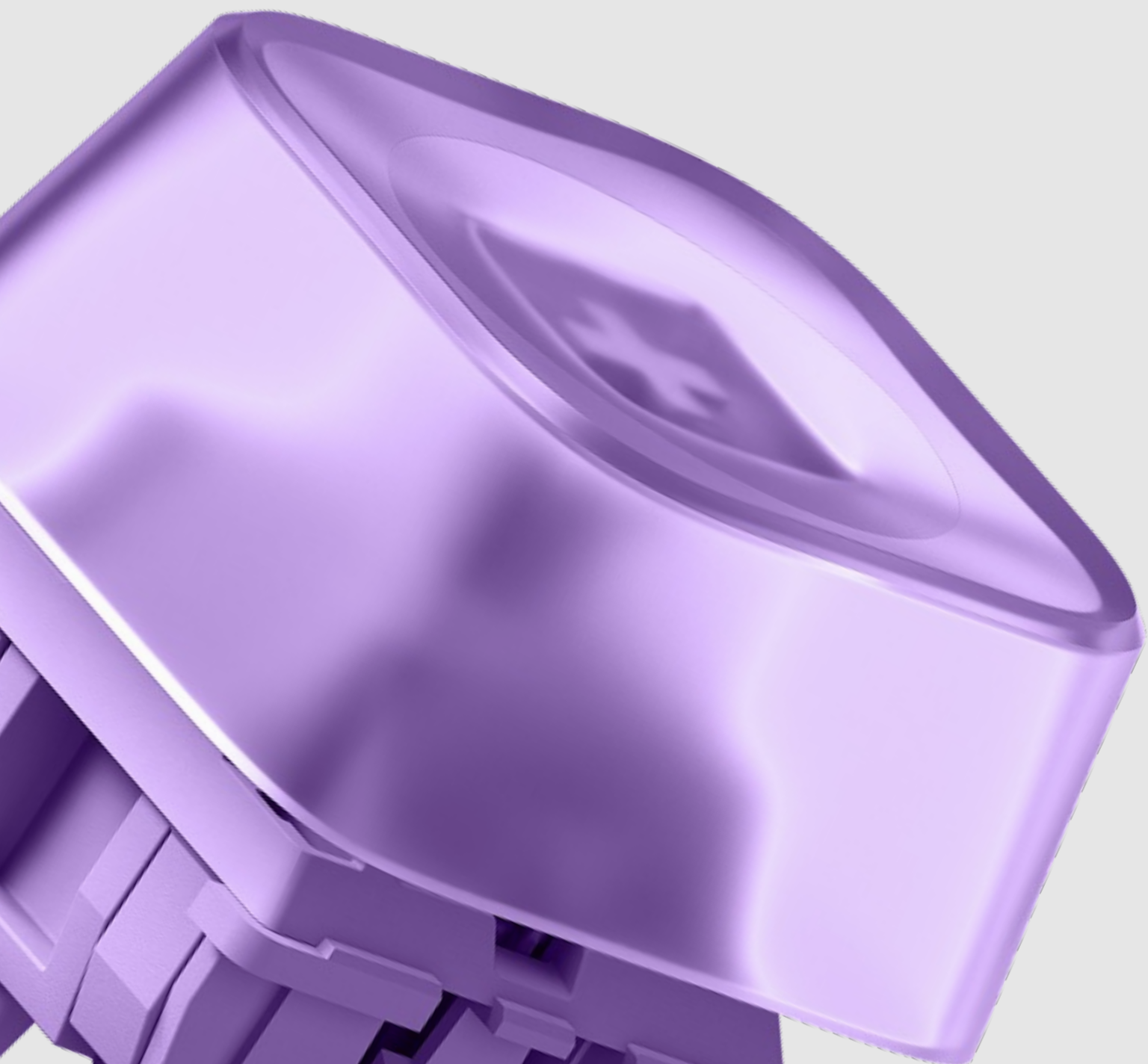
Специальные символы \$\$ используются вместо кавычек, чтобы код было удобно читать.





# Создание и использование функций (UDF)

## Часть 2



→ Базовый синтаксис CREATE FUNCTION

→ Режимы параметров: IN, OUT, INOUT

→ Возврат скаляров и таблиц (SETOF, TABLE)

→ Перегрузка функций



# Базовый синтаксис CREATE FUNCTION

```
CREATE OR REPLACE FUNCTION
имя_функции(параметр1 тип, параметр2
тип)
RETURNS тип_результата AS $$
BEGIN
    -- Логика
    RETURN результат;
END;
$$ LANGUAGE plpgsql;
```

Команда CREATE OR REPLACE FUNCTION — стандарт де-факто (создаёт новую или обновляет существующую).

Обязательно нужно указать следующую информацию.

1. Имя функции и аргументы
2. Тип возвращаемого значения (RETURNS ...)
3. Тело функции (внутри \$\$ ... \$\$)
4. Язык написания (LANGUAGE plpgsql)

# Пример: расчёт стоимости с налогом

```
CREATE OR REPLACE FUNCTION calculate_total(price NUMERIC, tax_rate NUMERIC)
RETURNS NUMERIC AS $$
DECLARE
    total NUMERIC;
BEGIN
    total := price + (price * tax_rate / 100);
    RETURN total;
END;
$$ LANGUAGE plpgsql;
```

Вызов функции: `SELECT calculate_total(1000, 20);` -- вернёт 1200

# Режимы параметров (IN, OUT, INOUT)

- **IN** (по умолчанию) — входной параметр. Внутри функции он доступен только для чтения (нельзя изменить его значение).
- **OUT** — выходной параметр. Используется, чтобы функция могла вернуть сразу несколько значений (без создания составного типа).
- **INOUT** — комбинированный. Принимает значение, может быть изменён внутри кода и возвращается как результат.

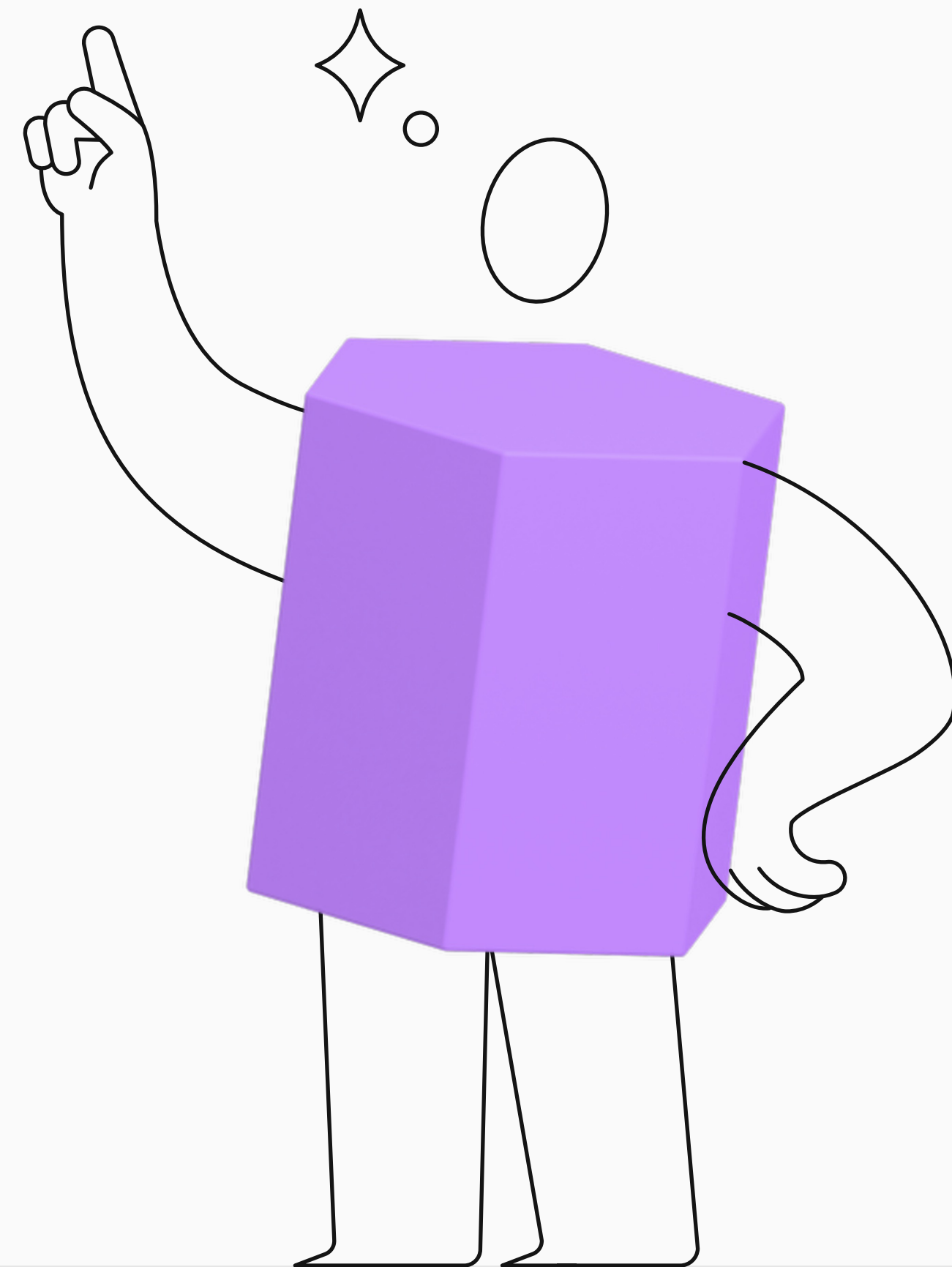
## Пример с OUT

```
CREATE FUNCTION get_min_max(IN a INT, IN  
b INT, OUT min_val INT, OUT max_val INT)  
...
```



# Параметры по умолчанию

- Делают вызов функции более гибким.
- Синтаксис: имя\_параметра тип DEFAULT значение (или просто = значение).
- Правило: параметры по умолчанию должны находиться в конце списка аргументов.
- Пример: `CREATE FUNCTION apply_discount(price NUMERIC, discount NUMERIC DEFAULT 5) ...`
- Вызов:
  - `SELECT apply_discount(1000);` – применит скидку 5%;
  - `SELECT apply_discount(1000, 15);` – применит скидку 15%.



# RETURNS TABLE

## Функции как динамические представления

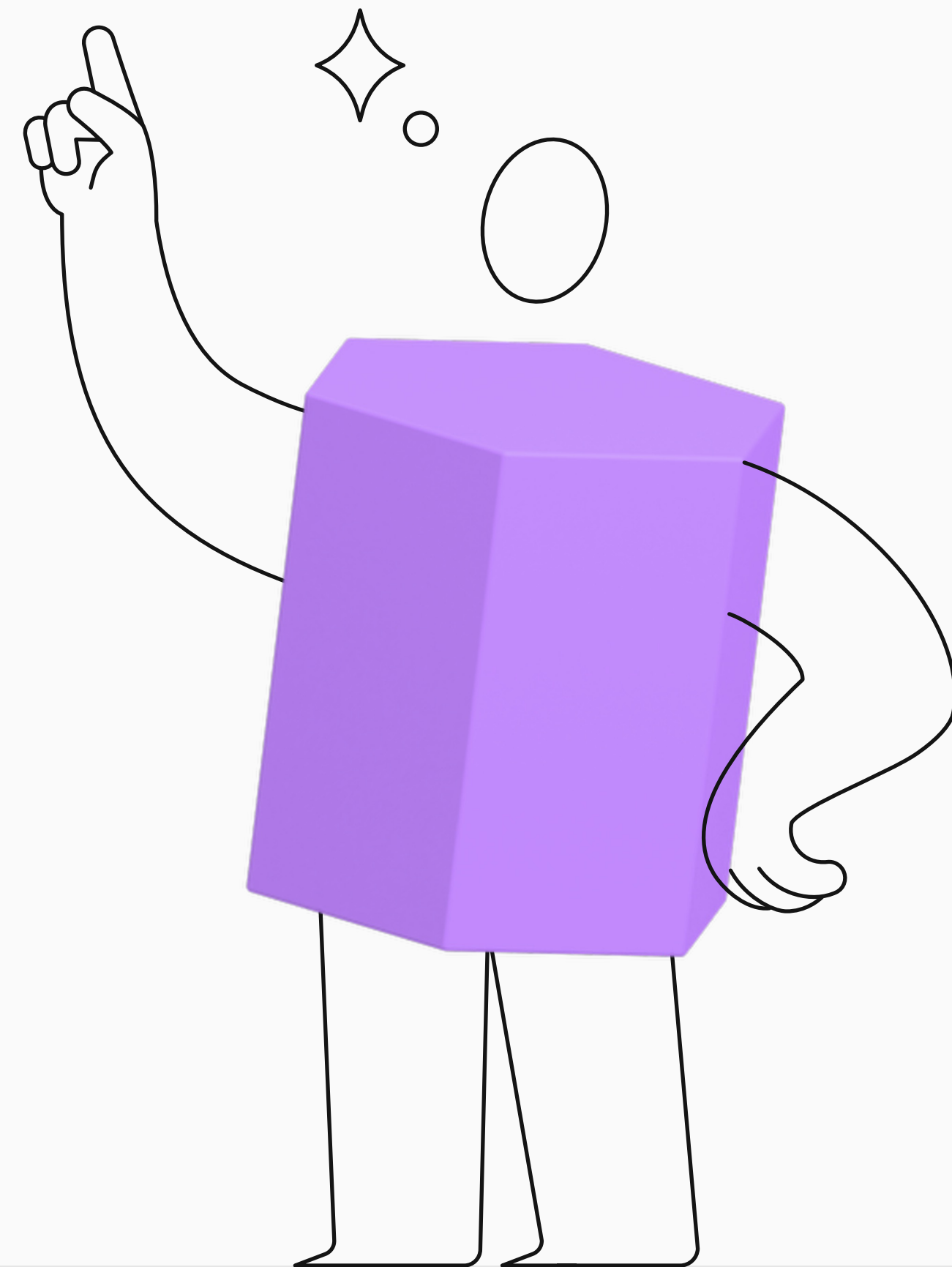
- Функции могут возвращать не только одно число или строку (скаляр), но и целые таблицы (наборы строк).
- Синтаксис: RETURNS TABLE (имя\_колонки тип, ...).
- Используется команда RETURN QUERY.

```
CREATE FUNCTION
get_users_by_status(p_status VARCHAR)
RETURNS TABLE (user_id INT, username
VARCHAR) AS $$
BEGIN
    RETURN QUERY
    SELECT id, name FROM users WHERE
status = p_status;
END;
$$ LANGUAGE plpgsql;
```



# Перегрузка функций (Function Overloading)

- В PostgreSQL можно создать несколько функций с одинаковым именем.
- Условие: у них должны быть разные типы или разное количество аргументов (разная сигнатура).
- СУБД сама выберет нужную версию при вызове (на основе переданных данных).
- Пример: две функции `get_user`.
  - `get_user(user_id INT)` — ищет по ID.
  - `get_user(email VARCHAR)` — ищет по почте.



# Процедурная логика и управление данными

## Часть 3

→ Переменные и типы данных

→ Условные конструкции (IF, CASE)

→ Виды циклов (LOOP, WHILE, FOR)

→ Сохранение результата запроса (SELECT INTO)

→ Динамический SQL (EXECUTE)

# Работа с переменными

Все переменные объявляются в блоке DECLARE.

Специальные типы данных (якорные типы):

- %TYPE — копирует тип данных конкретного столбца таблицы;
- %ROWTYPE — копирует структуру целой строки таблицы.

## Пример

DECLARE

`v_counter INT := 0; — обычная переменная.`

`v_user_email users.email%TYPE; — тип как у колонки email.`

`v_user_row users%ROWTYPE; — переменная, хранящая целую строку`  
BEGIN.

`v_counter := v_counter + 1; — присваивание.`



# Логика ветвления: IF и CASE

IF

```
IF v_balance >= 1000 THEN
    status := 'VIP';
ELSIF v_balance > 0 THEN
    status := 'Active';
ELSE
    status := 'Blocked';
END IF;
```

Case

```
CASE v_role
    WHEN 'admin' THEN
        -- дать все права
    WHEN 'user' THEN
        -- дать права на чтение
    ELSE
        -- отказать в доступе
END CASE;
```

# Циклы в PL/pgSQL

LOOP

LOOP

    v\_count := v\_count +  
1;

    EXIT WHEN v\_count >  
10;

END LOOP;

WHILE

WHILE v\_count < 10 LOOP

    ...  
END LOOP;

FOR

FOR i IN 1..10 LOOP

    ...  
END LOOP;

# Цикл по строкам таблицы

```
DECLARE
    v_record RECORD;
BEGIN
    FOR v_record IN (SELECT id, amount FROM orders WHERE status = 'pending')
    LOOP
        -- Обработка каждой строки
        RAISE NOTICE 'Обрабатываем заказ % на сумму %', v_record.id,
v_record.amount;
    END LOOP;
END;
```

Специальный вид цикла FOR, который перебирает результаты SQL-запроса построчно. Автоматически создает переменную-запись (RECORD).



# Сохранение данных в переменные и динамический SQL

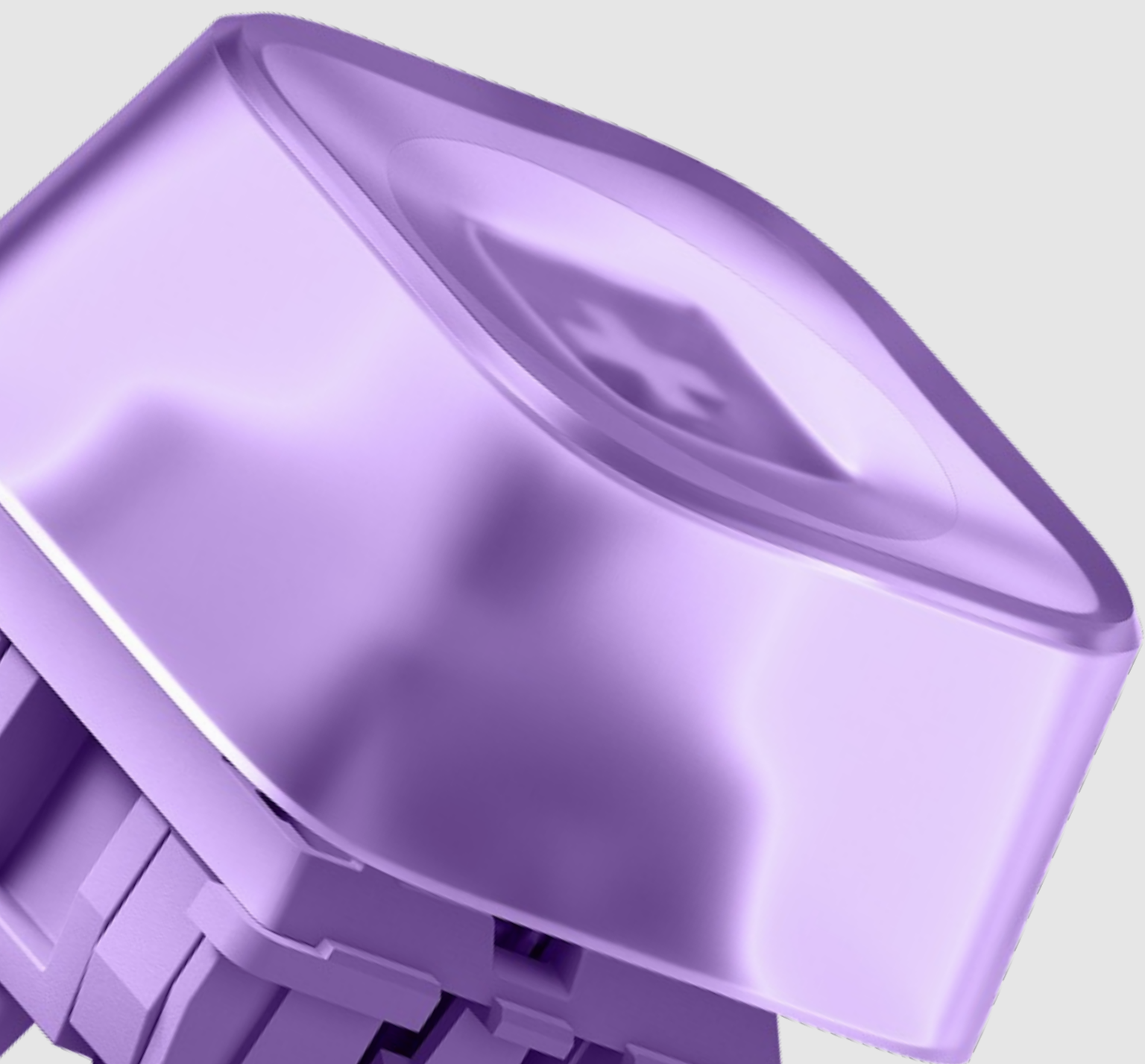
```
SELECT INTO
-- Ожидается, что запрос вернёт ровно 1
строку
SELECT balance INTO v_current_balance
FROM users WHERE id = 1;
```

```
EXECUTE
v_table_name := 'users_2023';
-- Строим строку с запросом и выполняем
её
EXECUTE 'SELECT count(*) FROM ' ||
v_table_name INTO v_count;
```

# Хранимые процедуры

Транзакции и отличия

Часть 4



→ Историческая справка: почему в PostgreSQL 11 появились процедуры

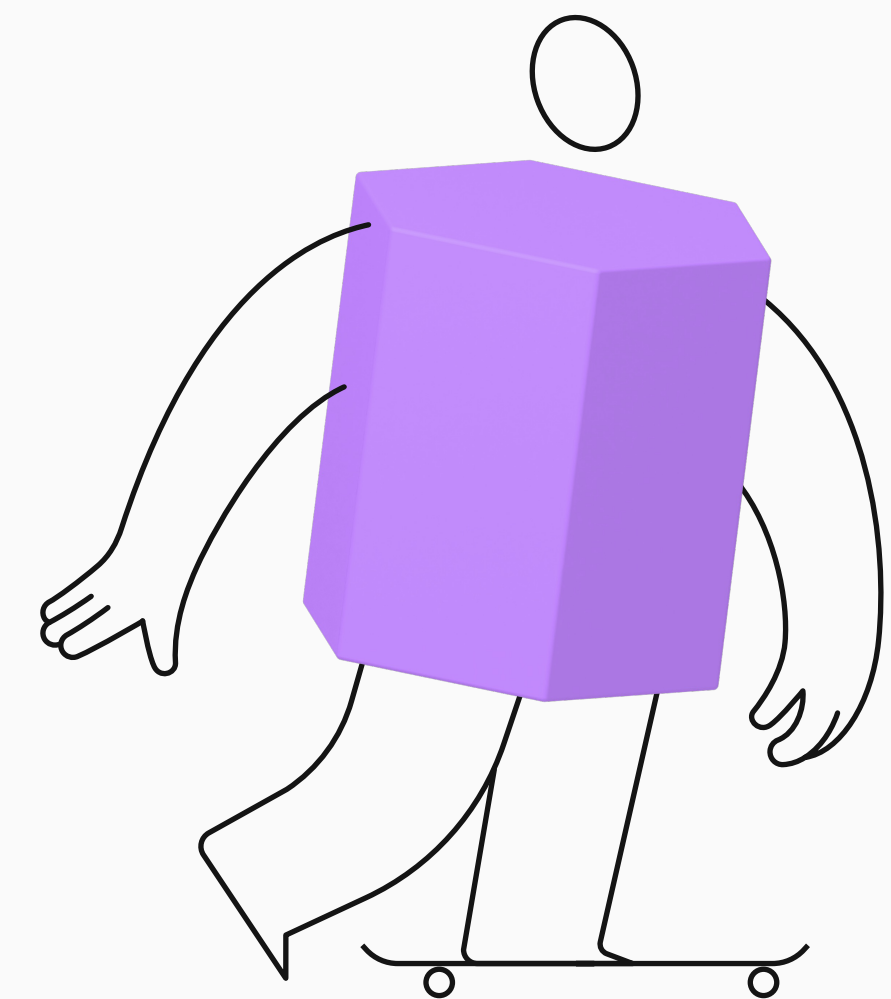
→ Функции vs процедуры: детальное сравнение

→ Синтаксис CREATE PROCEDURE и вызов через CALL

→ Управление транзакциями внутри кода (COMMIT и ROLLBACK)

# Зачем нужны процедуры, если есть функции?

- До PostgreSQL 11: существовали только функции (CREATE FUNCTION). Их часто неформально называли «хранимыми процедурами», но технически это было неверно.
- Главная проблема функций: функция всегда выполняется в рамках одной гигантской транзакции.
- Если функция обрабатывает 1 миллион строк и на 999 999-й строке происходит ошибка — откатятся (Rollback) абсолютно все изменения.
- Решение (с 11 версии): введение полноценных процедур (CREATE PROCEDURE), которые умеют разбивать работу на несколько независимых транзакций.





# Функция vs процедура: главные отличия

| Критерий         | Функция                             | Процедура                            |
|------------------|-------------------------------------|--------------------------------------|
| Создание         | CREATE FUNCTION                     | CREATE PROCEDURE                     |
| Вызов            | Внутри запроса: SELECT func()       | Отдельной командой: CALL proc()      |
| Возврат значения | Обязана вернуть результат (RETURNS) | Не возвращает значение (нет RETURNS) |
| Транзакции       | Нельзя делать COMMIT/ROLLBACK       | Можно делать COMMIT/ROLLBACK         |

# Синтаксис создания и вызова процедуры

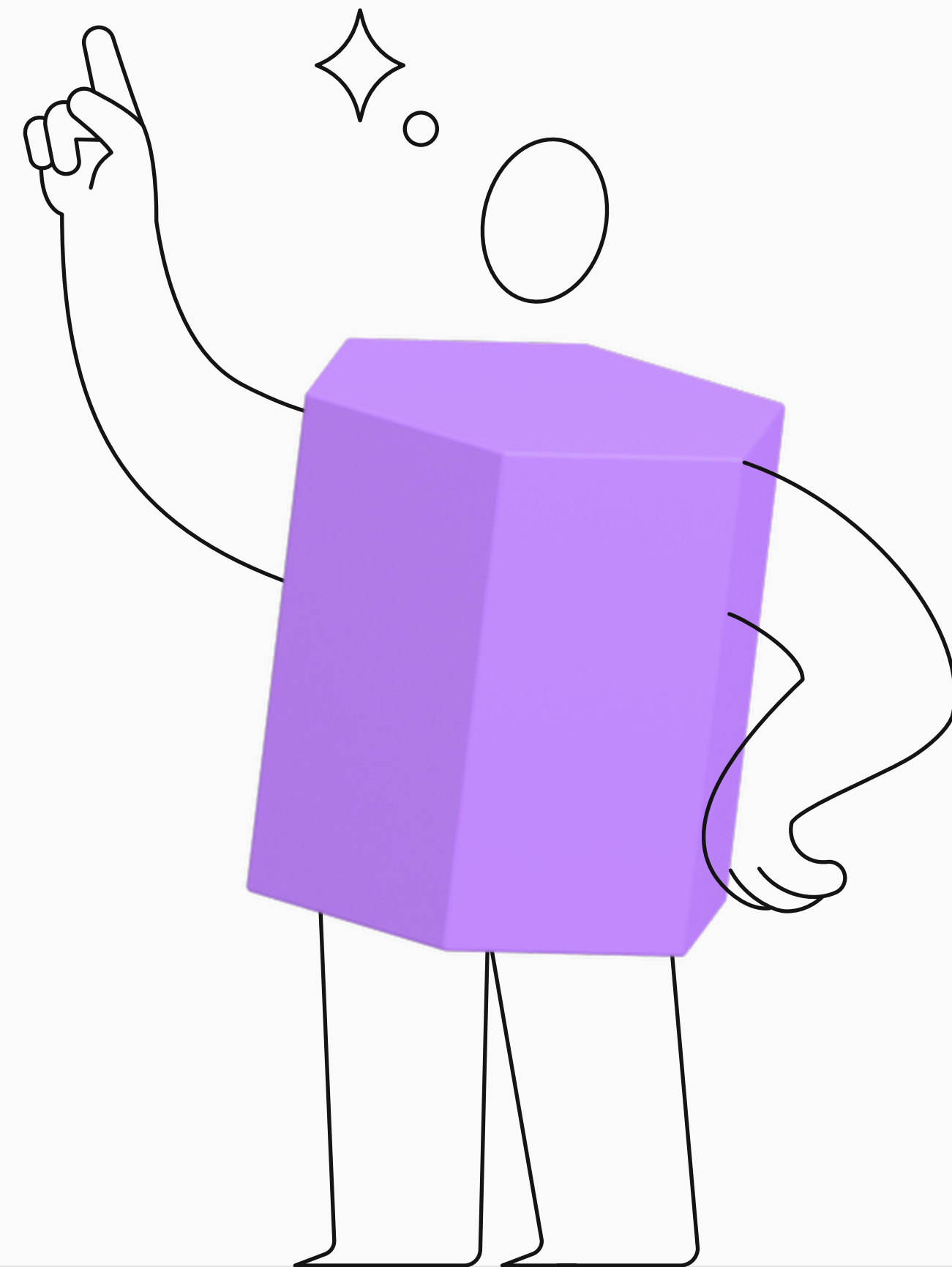
```
CREATE OR REPLACE PROCEDURE archive_old_logs(p_year INT)
LANGUAGE plpgsql AS $$
BEGIN
    -- Переносим данные в архив
    INSERT INTO logs_archive SELECT * FROM logs WHERE year < p_year;

    -- Удаляем старые данные
    DELETE FROM logs WHERE year < p_year;
END;
$$;
```

Вызов: `CALL archive_old_logs(2022);`

# Управление транзакциями (TCL)

- COMMIT — фиксирует (сохраняет) все изменения, сделанные с момента начала процедуры (или прошлого коммита), и освобождает блокировки.
- ROLLBACK — отменяет изменения.
- Зачем это нужно? Для пакетной обработки (Batch processing). Можно обрабатывать данные по 10 000 строк и делать COMMIT. Это спасёт оперативную память сервера.





# Пример: банковский перевод (Классика)

```
CREATE PROCEDURE transfer_funds(sender INT, receiver INT, amount NUMERIC)
LANGUAGE plpgsql AS $$
BEGIN
    -- Снимаем деньги с отправителя
    UPDATE accounts SET balance = balance - amount WHERE id = sender;

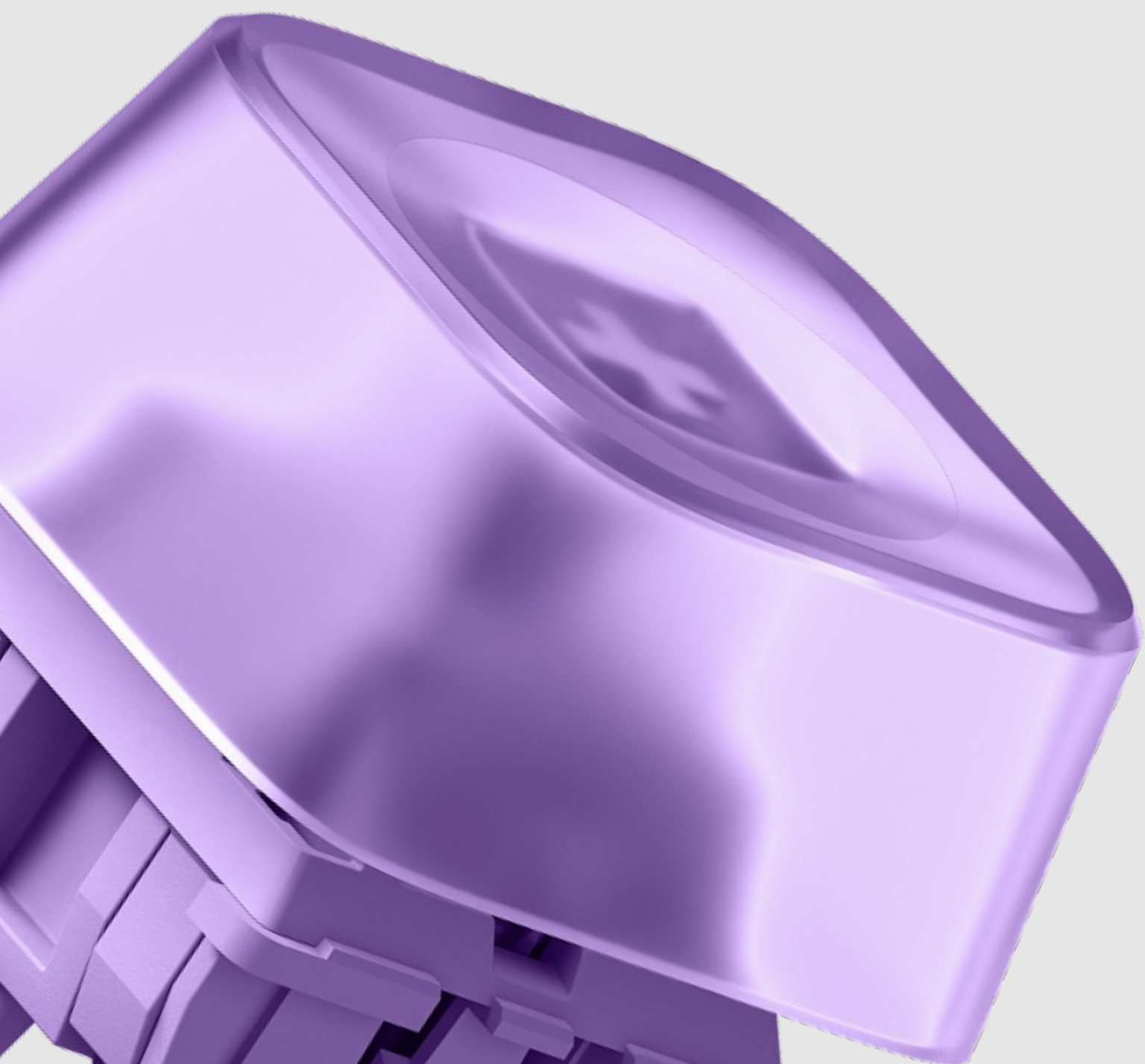
    -- Зачисляем получателю
    UPDATE accounts SET balance = balance + amount WHERE id = receiver;

    -- Фиксируем успешный перевод!
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        -- Если что-то пошло не так (например, нет денег), отменяем всё
        ROLLBACK;
        RAISE NOTICE 'Ошибка перевода. Транзакция отменена.';
END;
$;
```



# Продвинутые концепции и оптимизация

## Часть 5



→ Категории изменчивости (Volatility):  
IMMUTABLE, STABLE, VOLATILE

→ Безопасность: SECURITY INVOKER vs SECURITY DEFINER

→ Подсказки планировщику: COST и ROWS

→ Параллельное выполнение функций



# Категории изменчивости (Volatility)

|             | IMMUTABLE                                                | STABLE                                                                                                       | VOLATILE                                                                    |
|-------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Пример      | <code>calculate_square_root(4)</code><br>всегда вернёт 2 | <code>get_current_usd_rate()</code><br>(курс меняется раз в день,<br>но в одной транзакции<br>он фиксирован) | <code>generate_random_number()</code> ,<br><code>increment_counter()</code> |
| Оптимизация | Результат может<br>быть кеширован<br>и переиспользован   | Результат может быть<br>кеширован в рамках запроса                                                           | Не кешируется, вызывается<br>при каждом обращении                           |

# SECURITY INVOKER vs SECURITY DEFINER

## SECURITY INVOKER (по умолчанию)

Функция выполняется с правами пользователя, который её вызвал.

Если пользователь `guest` не имеет доступа к таблице `salaries`, то функция, вызванная им, тоже не сможет прочитать эти данные.

**Безопасность:** высокая (каждый работает под своими правами)

## SECURITY DEFINER

Функция выполняется с правами пользователя, который её создал.

Позволяет предоставить ограниченный доступ к защищенным данным: создать функцию `get_own_salary(user_id INT)` под правами `admin`, дать `guest` право только на выполнение этой функции.

**Опасность:** это мощный инструмент, но требует осторожности! Нельзя допускать SQL-инъекций в таких функциях

# Подсказки планировщику (COST и ROWS)

## COST (по умолчанию 100)

Приблизительная стоимость выполнения функции относительно «простого» SQL-оператора.

COST 1 — очень лёгкая. COST 10000 — очень тяжёлая.

**Влияние:** планировщик будет стараться выполнять «дорогие» функции как можно реже или позже

## ROWS (по умолчанию 1000)

Только для функций, возвращающих таблицы (RETURNS TABLE или SETOF).

Приблизительное количество строк, которые функция вернёт.

**Влияние:** помогает планировщику строить оптимальные планы JOIN и упорядочивания данных

## Пример

```
CREATE FUNCTION
my_heavy_function()
RETURNS TABLE (...) AS $$
... $$
LANGUAGE plpgsql

COST 5000 ROWS 10; --
Очень дорогая, но вернёт
мало строк
```



# Параллельное выполнение функций (Parallel Safety)

## PARALLEL UNSAFE

Не может быть безопасно выполнена в параллельном запросе.

**Пример:** функции, которые изменяют данные (UPDATE, INSERT), генерируют случайные числа (не детерминированы).

**Влияние:** запрос будет выполнен в один поток

## PARALLEL RESTRICTED

Может быть параллельной, но с ограничениями (например, не может запускаться в параллельных worker).

**Пример:** функции, которые читают системные каталоги

## PARALLEL SAFE

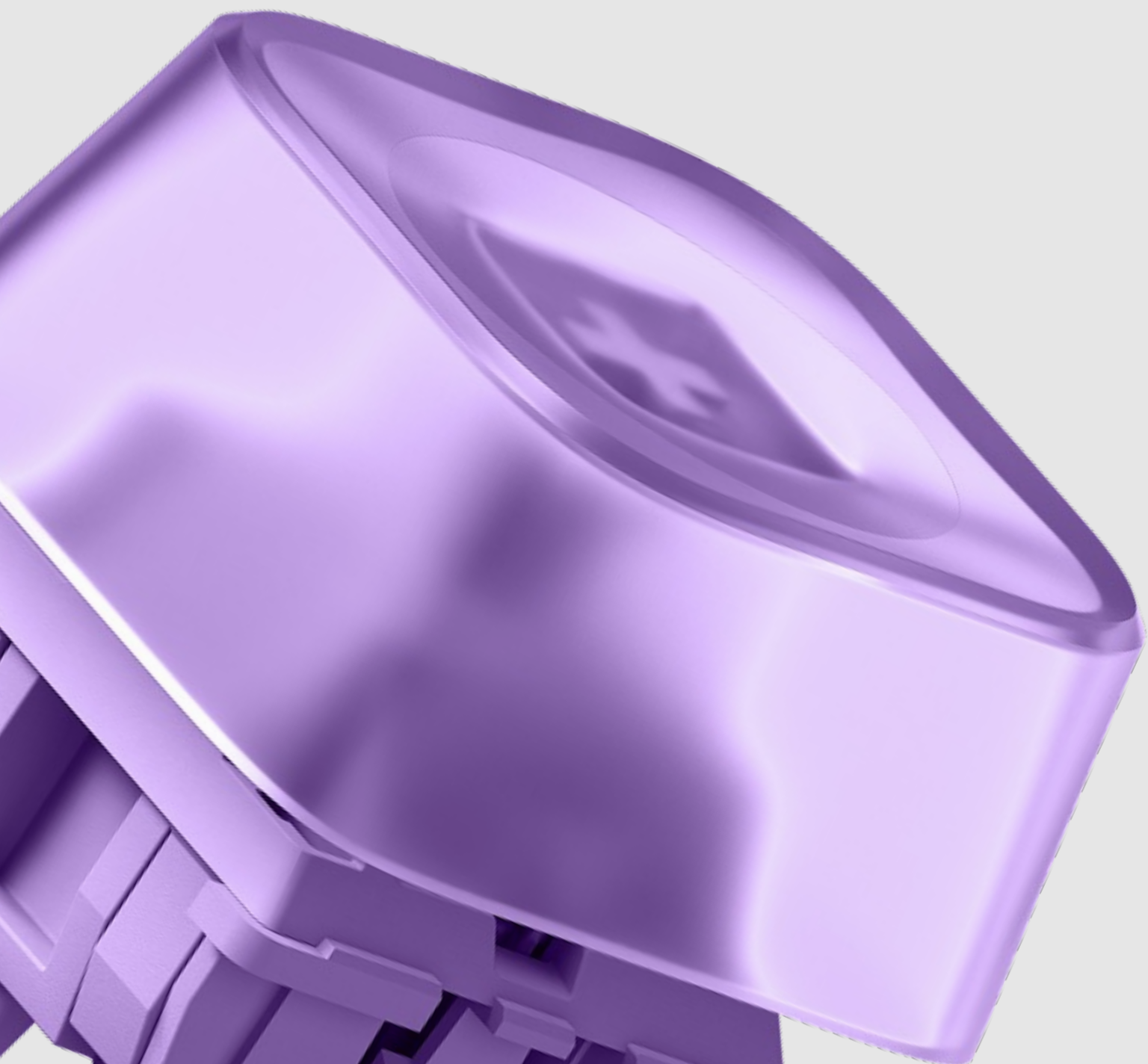
Полностью безопасно может быть запущена в любом параллельном контексте.

**Пример:** функции, которые только читают данные, не меняют состояние, не вызывают внешние функции.

**Влияние:** запрос может быть существенно ускорен

# Специальные случаи применения

## Часть 6



→ Триггерные функции: автоматизация действий

→ Пользовательские агрегаты (UDAF): пишем свой SUM()

→ Рекурсия: работа с деревьями и иерархиями

→ Полиморфные типы (anyelement): универсальный код

# Триггерные функции

**Особенность PostgreSQL:** в отличие от других СУБД, здесь логика триггера не пишется внутри самого триггера. Сначала создаётся специальная функция, возвращающая тип TRIGGER, а затем она привязывается к событию (INSERT, UPDATE, DELETE).

## Специальные переменные:

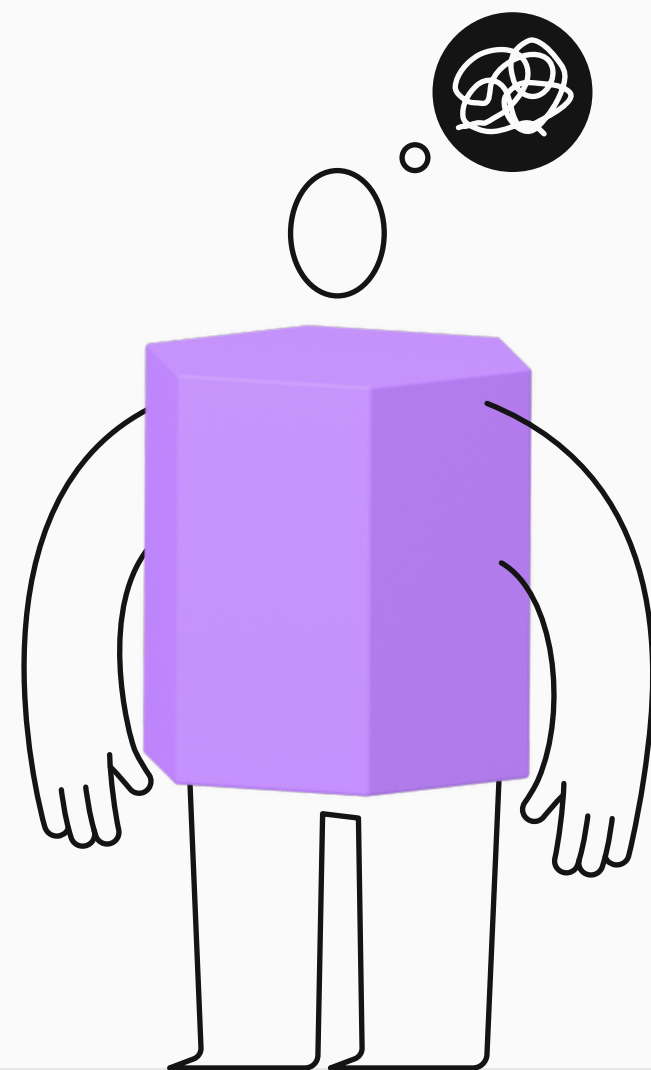
- NEW — содержит новую строку (например, данные, которые пытаются вставить);
- OLD — содержит старую строку (данные до обновления или удаления).



# Пользовательские агрегатные функции (UDAF)

Стандартные агрегаты: SUM(), AVG(), MAX().

Но что, если нам нужно перемножить все числа в столбце? Или посчитать сложную медиану?



Как это работает в Postgres

1. Пишется функция перехода состояния (SFUNC — State Function), которая принимает текущее накопленное значение и новую строку, возвращая обновлённое состояние.
2. Создаётся агрегат командой CREATE AGGREGATE.

Это позволяет обрабатывать миллионы строк без выгрузки их в приложение.

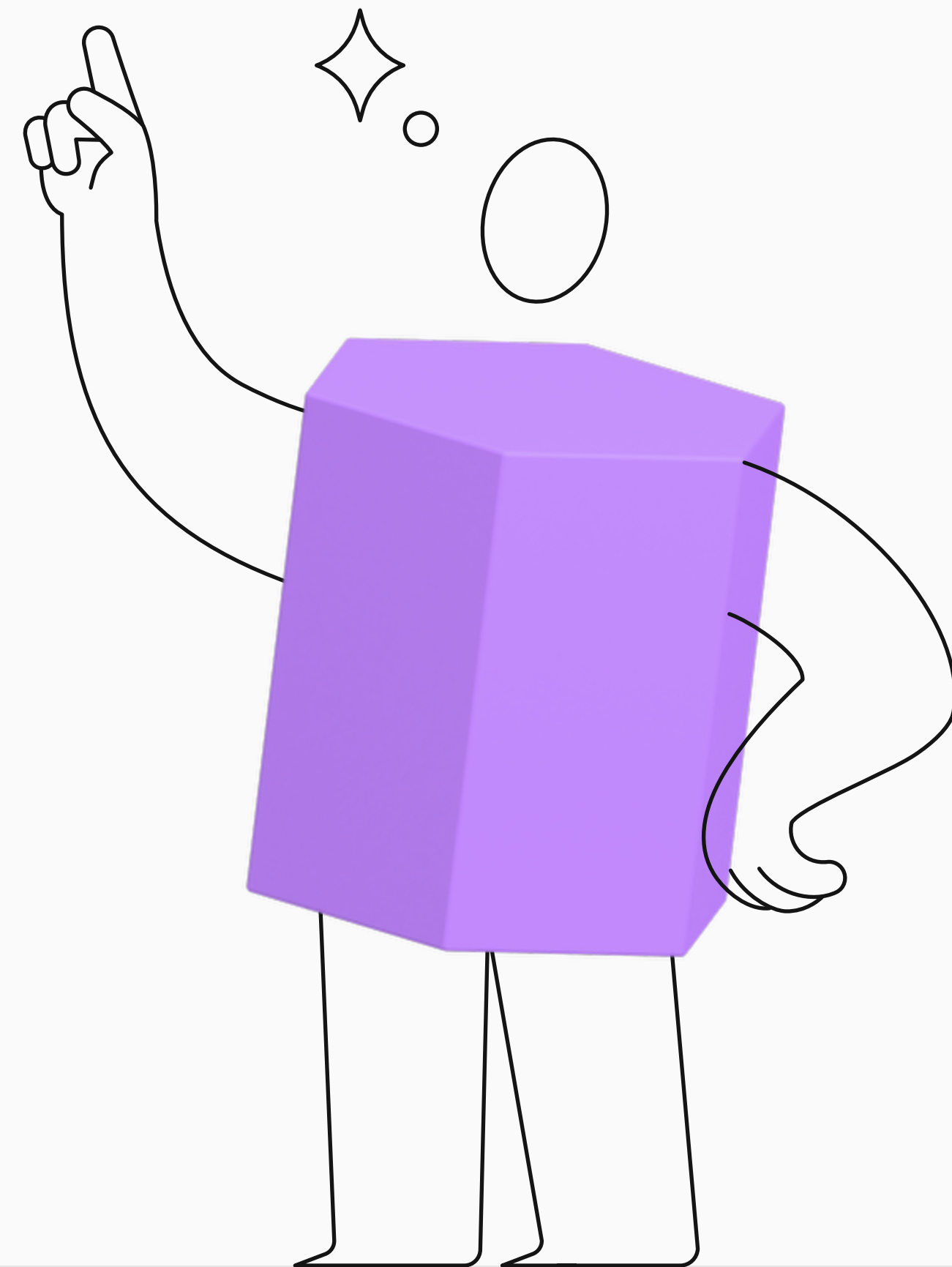


# Рекурсивные функции

**Зачем нужна:** идеальна для работы с иерархиями (дерево комментариев, структура подчинения сотрудников, категории товаров).

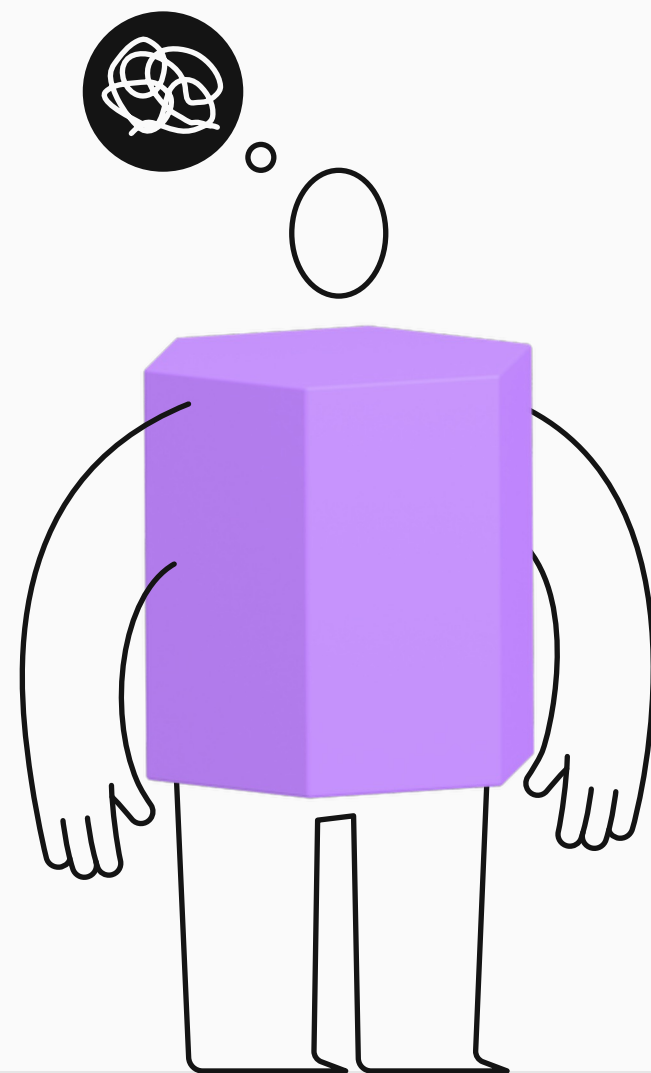
**Альтернатива:** рекурсивные CTE (WITH RECURSIVE). Функции используются, когда при обходе дерева нужна сложная процедурная логика (условия, обновления).

**Важное правило:** обязательно должно быть условие выхода (Base Case), иначе функция уйдёт в бесконечный цикл и исчерпает память (Stack Overflow).



# Полиморфные типы данных (дженерики)

Псевдотипы `anyelement` (любой элемент) и `anyarray` (массив любых элементов).

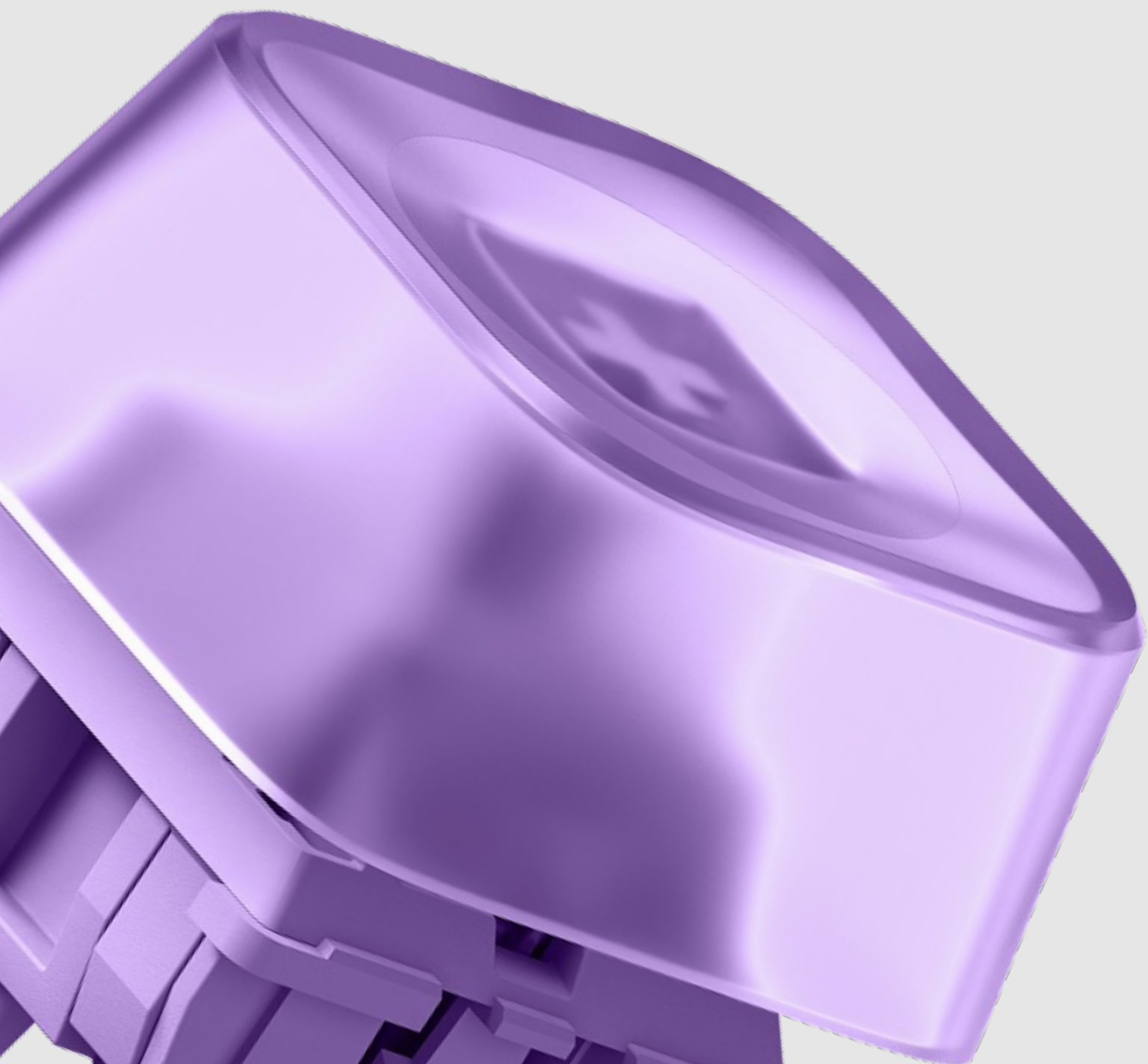


Пример

```
CREATE FUNCTION get_max(a anyelement,  
b anyelement)  
RETURNS anyelement AS $$ ... $$
```

# Безопасность, отладка и архитектура

## Часть 7



→ Обработка исключений и логирование (RAISE)

→ Защита от SQL-инъекций в динамическом SQL

→ Внутреннее устройство: где живут функции (pg\_proc)

→ Архитектурный холивар: «толстая» vs «тонкая» база данных



# Обработка ошибок и логирование

**Логирование (RAISE):** уровни сообщений (DEBUG, NOTICE, WARNING, EXCEPTION).

**Пример:** RAISE NOTICE 'Обработан пользователь ID: %', user\_id;

RAISE EXCEPTION — принудительно прерывает выполнение (выбрасывает ошибку).



Блок EXCEPTION: перехват системных или пользовательских ошибок, чтобы предотвратить падение всей транзакции.

**Пример**

**BEGIN**

**-- Опасный код (например, деление на ноль)**

**EXCEPTION**

**WHEN division\_by\_zero THEN**

**RAISE WARNING 'Попытка деления на ноль игнорируется';**

**RETURN 0;**

**END;**

# Кибербезопасность: SQL-инъекции

При склеивании строк для EXECUTE злоумышленник может передать вредоносный код.

Решение — функция format():

- %I — экранирует идентификаторы (имена таблиц, колонок);
- %L — экранирует литералы (значения).

Плохо



```
EXECUTE 'SELECT * FROM users WHERE  
name = ''' || p_name || ''';
```

Хорошо



```
EXECUTE format('SELECT * FROM %I WHERE  
name = %L', p_table, p_name);
```

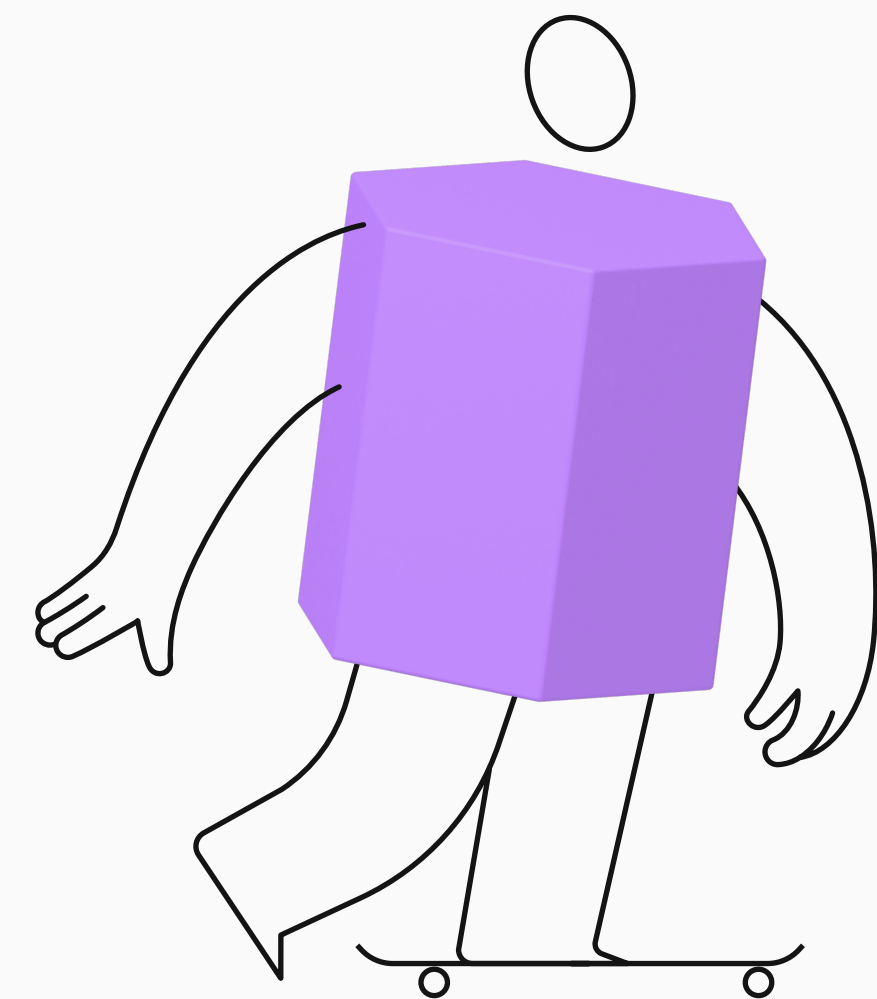
# Внутреннее устройство базы данных

Системный каталог `pg_proc` хранит метаданные и исходный код всех функций и процедур (как встроенных, так и пользовательских).

Полезный запрос для аудита (показать код функции)

```
SELECT proname AS Имя, prosrc AS Код  
FROM pg_proc  
WHERE proname = 'calculate_total';
```

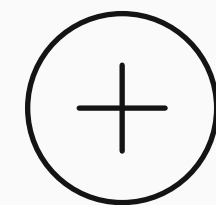
Таблица `pg_depend` отслеживает зависимости (почему нельзя удалить таблицу, если её использует функция).



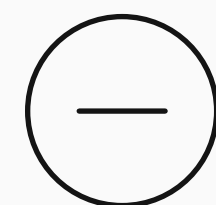


# Итоги: «толстая» vs «тонкая» база данных

## Толстая БД



Максимальная скорость, защита данных (приложение не имеет прямого доступа к таблицам, только через вызовы функций).



Труднее тестировать (Unit-тесты в SQL сложнее), тяжело масштабировать сервер БД, дорогой поиск разработчиков

## Тонкая БД

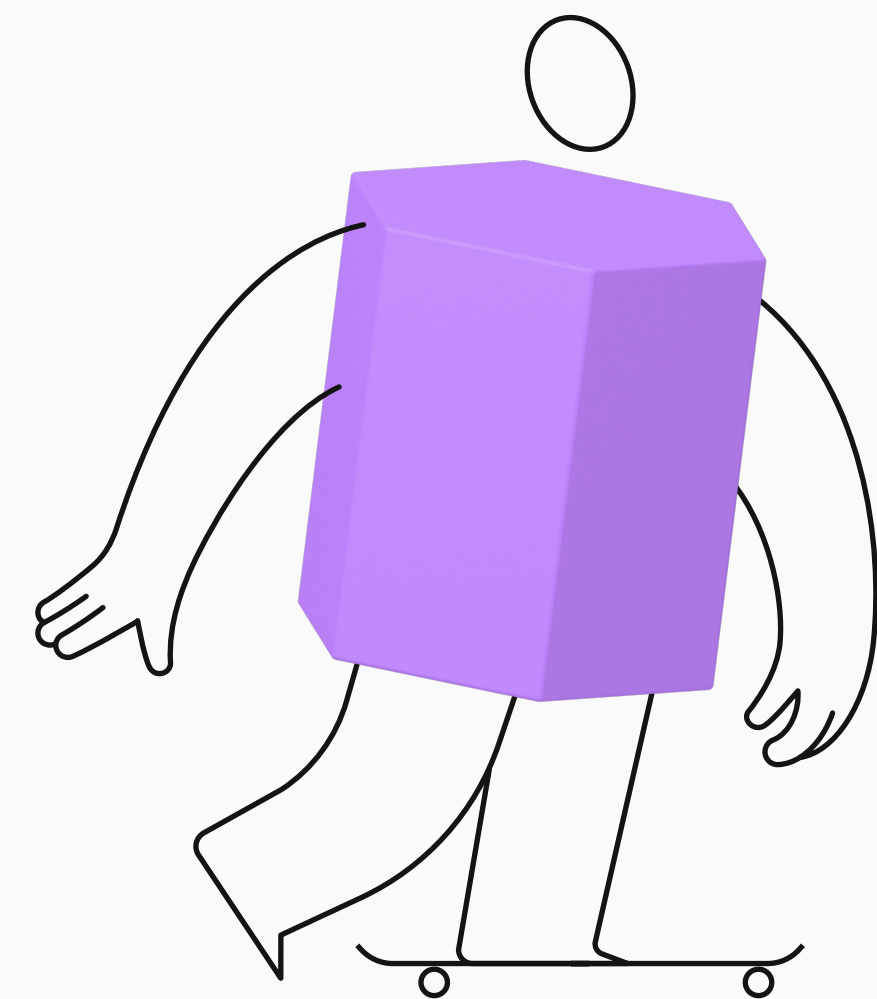
Вся логика в приложении (Python/Java), легко масштабировать добавлением новых серверов приложения

Сетевые задержки (гоняются мегабайты данных туда-сюда), риск рассинхронизации логики, если к БД подключены разные приложения

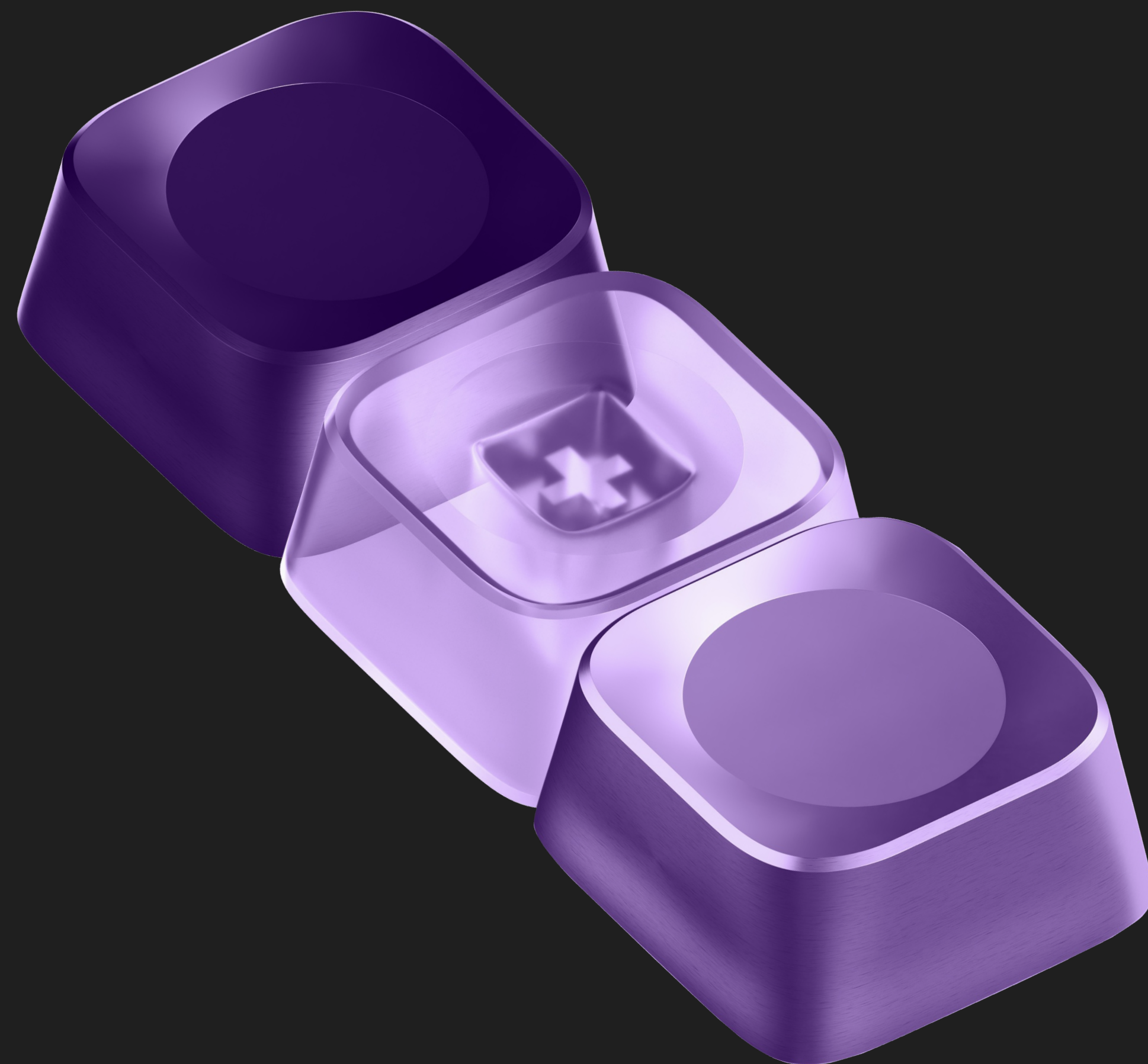


# Заключение

1. PL/pgSQL превращает PostgreSQL из хранилища в вычислительную платформу.
2. Функции (SELECT) — для вычислений, процедуры (CALL) — для автоматизации процессов с транзакциями.
3. Умелое использование индексов изменчивости (IMMUTABLE/STABLE) и параметров стоимости (COST) ускоряет работу системы в разы.
4. Безопасность (защита от инъекций и управление правами) должна закладываться на этапе написания кода.



# Вопросы



## Минутка обратной связи

